Cross-Workload Flow-Size Prediction for AI Data Center Networks

Leonardo Alberro*†, Pedro Casas†, Matías Richart*, Eduardo Grampin*
*Universidad de la República, Montevideo, Uruguay †AIT Austrian Institute of Technology, Vienna, Austria
{lalberro, grampin, mrichart}@fing.edu.uy, pedro.casas@ait.ac.at

***Abstract*—Modern congestion control mechanisms for data center networks (DCNs) increasingly depend on early knowledge of flow size to differentiate latency-sensitive short flows from throughput-oriented long flows. While AI workloads exhibit recurring communication patterns that make flow-size prediction feasible, existing approaches either rely on unrealistic oracle assumptions or require workload-specific models that do not scale across diverse applications. This paper presents a cross-workload flow-size prediction framework as a practical foundation for learning-assisted flow-aware congestion control. We develop a unified predictor that operates across heterogeneous AI workloads by aligning partially overlapping end-host telemetry into a common feature space. We evaluate the predictor from both machine-learning and networking perspectives, combining standard regression metrics with transport-oriented measures such as threshold-crossing errors, short/long flow misclassification, and end-to-end flow completion time (FCT) in a state-of-the-art flow-aware transport protocol. Our results show that the proposed predictor achieves over 98% short/long flow classification accuracy while preserving most of the predictive quality of workload-specific models and maintaining comparable transport performance. These results demonstrate that accurate, deployable flow-size prediction can be achieved without oracle flow-size knowledge or maintaining a separate model for each workload. To foster transparency and reproducibility, we release the datasets, trained models, and simulation modules.**

***Index Terms*—Data Center Networks, Transport Protocols, Machine Learning.**

I. Introduction

Data center networks (DCNs) support latency-sensitive and bandwidth-intensive applications at a massive scale. In these environments, transport-layer performance is strongly influenced by how congestion control mechanisms react to heterogeneous traffic demands. Although TCP remains the dominant transport protocol, its behavior is known to be suboptimal in DCNs, where shallow buffers, high fan-in communication patterns, and low-latency requirements expose considerable limitations [1]. These directly affect flow completion time (FCT), throughput, and overall resource efficiency.

A central observation in DCN transport design is that not all flows should be treated equally. Short flows are typically latency-sensitive and are disproportionately affected by queueing delays, whereas long flows dominate byte volume and primarily determine throughput and buffer occupancy. This asymmetry has motivated several flow-aware congestion control mechanisms [2]–[4], which attempt to distinguish short and long flows and assign them different transport behaviors. The effectiveness of such mechanisms, however, depends on the ability to identify the flow class early enough to influence congestion control decisions.

This distinction is becoming even more important in modern AI data centers, where large-scale distributed training and inference workloads impose strict performance and efficiency requirements. Here, even small communication inefficiencies can translate into increased job completion time, reduced accelerator utilization, and higher operational cost [5]. At the same time, AI workloads often exhibit more repetitive and structured communication behavior than traditional cloud applications [6], [7]. This regularity creates an opportunity to improve transport decisions through early flow-size estimation.

Despite this opportunity, obtaining flow-size information in practice remains challenging. Some flow-aware transport mechanisms assume that the exact flow size is available before transmission, for example, through application-provided information [3]. While useful as an upper bound, this assumption is difficult to satisfy in general-purpose deployments and limits the practical applicability of such mechanisms. Other approaches infer flow behavior indirectly from signals such as TCP buffer occupancy or the number of bytes already transmitted [8]. These methods are easier to deploy, but they provide information only after the flow has already started, reducing their impact on early transport decisions.

Previous work has explored the use of machine learning to estimate flow size from historical traces [9], [10], showing that flow-size prediction is feasible, particularly for structured AI workloads. However, most existing models are trained and evaluated on individual out-of-date workloads. While workload-specific predictors can achieve high accuracy under stable conditions, they introduce practical deployment challenges: operators must train, maintain, and select among multiple models, and these models may generalize poorly to unseen workloads or mixed traffic conditions.

The challenge is further complicated by feature heterogeneity. Different AI training frameworks may expose different telemetry signals, including host-level statistics, application-level counters, temporal features, or network-level measurements. As a result, datasets collected from different training frameworks often do not share the same feature space, making it difficult to train a single predictor by simply merging traces. This raises a key question for deployable learning-assisted transport: can flow-size prediction be formulated across workloads in a way that preserves predictive quality while avoiding the complexity of maintaining one model per application?

In this paper, we address this question by studying cross-workload flow-size prediction for AI data center networks. Our goal is not only to improve prediction accuracy, but also to evaluate whether a unified predictor can provide sufficiently reliable flow-size information for flow-aware congestion control. To this end, we formulate the prediction problem across multiple workloads with heterogeneous feature spaces and investigate feature-alignment strategies that enable a single model to learn from diverse telemetry inputs. We then evaluate the resulting predictors at two levels. First, we assess their predictive quality using standard regression metrics and transport-relevant decision metrics, including threshold-crossing errors and short/long flow misclassification. Second, we integrate the predicted flow sizes into Fork [3], a state-of-the-art flow-aware transport mechanism that by default assumes oracle knowledge of the flow size, and quantify the resulting impact on FCT. The results show that a single cross-workload model can replace workload-specific models with limited loss in exact regression quality for most workloads. When integrated into Fork, the predictors retain a substantial fraction of the oracle-assisted transport benefit. The specific contributions of this paper are as follows:

- **Cross-workload flow-size prediction:** we introduce a unified learning framework for flow-size prediction across heterogeneous AI workloads, together with a feature-alignment methodology that enables a single model to learn from partially overlapping end-host telemetry spaces.
- **Deployable learning-assisted flow awareness:** we demonstrate that a unified predictor can replace workload-specific models and oracle flow-size knowledge with only limited degradation, providing a practical foundation for deployable learning-assisted flow-aware transport.
- **Transport-oriented evaluation:** we present a multi-level evaluation methodology that bridges prediction quality and transport performance, combining regression metrics, threshold-crossing errors, short/long flow classification, and end-to-end validation through integration with the Fork congestion-control protocol.
- **Transport-layer insights:** we quantify how different prediction errors affect FCT, showing that underestimating large flows is substantially more harmful than overestimating short flows and identifying the transport-relevant error characteristics that should guide future predictors.
- **Open datasets and reproducibility:** we extend existing public datasets with updated distributed Spark workloads and a GPT fine-tuning workload, providing modern per-flow and end-host telemetry traces together with models and evaluation artifacts that will be released to the community [11].

II. Background and Related Work

Modern distributed training frameworks often reuse persistent TCP connections to avoid connection setup overhead and to support frequent exchanges of small messages. In such settings, a long-lived TCP connection identified by the standard 5-tuple may carry multiple logically distinct bursts, making the notion of a flow too coarse. Following prior work [9], [10], we define a flow as a burst of packets exchanged between two hosts and separated from other bursts by an inactivity gap, i.e., a *flowlet*. This abstraction better captures the communication units relevant to transport performance. Accordingly, FCT remains our primary metric, as it measures the time required to complete each burst.

Flow-Aware Transport in DCNs. In DCN transport protocols, short and long flows impose different performance requirements. This asymmetry has motivated several DCN congestion control mechanisms that attempt to differentiate traffic according to flow size [2]–[4].

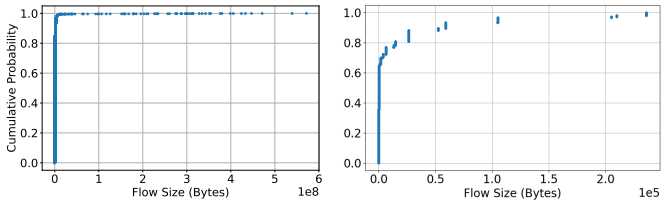
These mechanisms differ in how they obtain flow-size information. For example, Fork [3] assumes that the exact flow size is available before transmission. This provides a useful upper bound, but relies on oracle-like information and requires an application–transport interface, limiting deployability. Other approaches estimate flow size reactively. For example, a flow-aware modification of DCTCP [2] initially treats flows as short and reclassifies them as long after a byte threshold is exceeded. While simple, this delays the decision and may be ineffective in high-speed DCNs, where many short flows complete within a few RTTs. Finally, TCP send-buffer occupancy has also been used as a coarse signal [8], [12], but it depends on application behavior, receiver dynamics, and network conditions, making it unreliable as a general early indicator of flow size.

These limitations motivate the need for early and realistic flow-size estimation mechanisms. In particular, flow-aware transport protocols such as Fork would benefit from replacing oracle flow-size knowledge with predictions that are available before or near the beginning of each flowlet.

Traffic Prediction and Flow-Size Estimation. Traffic prediction has long been studied in network management using both statistical models [13] and deep learning techniques [14], [15]. However, most prior work forecasts aggregate demand over time at the link, path, or network level. In contrast, we estimate the size of individual flows or flowlets before transport decisions are made, a finer-grained problem that is directly relevant to flow-aware congestion control.

Machine learning offers a promising approach to this problem. Network transmissions are ultimately triggered by application behavior, computation, memory accesses, storage operations, and previous communication events. System and network traces can therefore provide predictive signals about future flow sizes. Prior work has used traces to learn flow-size information [9], [10], evaluating models such as LSTMs, gradient-boosted decision trees, and feed-forward neural networks. Other studies classify flows into mice and elephant using ML techniques [16]. However, these approaches are largely workload-specific, requiring separate models to be trained, selected, and maintained for each workload. Moreover, prior evaluations mostly focus on prediction metrics, without explicitly linking prediction errors to transport-level outcomes such as FCT.

AI DCN Workloads and Feature Heterogeneity. Prior DCN studies report, for example, that in representative data-mining workloads more than 80% of flows are smaller than 10 KB,



(a) Spark Page Rank. (b) TensorFlow.

Fig. 1: Flow size CDF for distributed AI workloads.

while large flows carry most of the traffic volume [1].

However, many public DCN traces are insufficient for learning-based flow-size prediction. Some provide only aggregate workload descriptions or flow-size distributions, while others lack the host-, application-, or system-level signals needed to predict individual flow sizes. For instance, Facebook's packet-sampled traces [6] characterize production traffic but do not expose the fine-grained per-flow and host-side context required for our task.

For this reason, datasets with both flow-level and host-level instrumentation are particularly relevant. The Flux [9] dataset includes distributed AI training traces, such as Spark PageRank, K-Means, SGD, and TensorFlow, enabling the study of flow-size predictability from temporal and system-level features. In addition to being out of date with modern AI training frameworks, these data also expose an important challenge: different workloads do not necessarily share the same feature space. Some workloads provide traces of temporal features, others expose host-level counters, and others provide only a subset of network or system statistics. Furthermore, their flow-size distributions may differ substantially. For instance, Figure 1 illustrates the flow-size distributions for Spark PageRank and TensorFlow. While Figure 1a shows that most flows are below 100 KB, the distribution for the TensorFlow workload (Figure 1b) concentrates most flows below 10 KB. Such differences complicate the direct application of a single learning model across workloads.

Position of This Work. Here, we exposed a key gap in current learning-based flow-size estimation: existing predictors are typically designed and evaluated on a per-workload basis. Although specialization can improve accuracy, it increases model-management complexity and is less practical when workloads evolve or expose different telemetry features. This limits the availability of early, realistic, and deployable flow-size estimation for congestion control. As a result, flow-aware transport still often relies on oracle flow sizes or signals unavailable at flow start. To address those gaps, we propose a unified flow-size predictor for heterogeneous features as a practical alternative to oracle-based flow awareness.

III. Predicting Flow Size from Measurements

We formulate flow-size prediction as a supervised regression problem over high-dimensional network telemetry, where the model estimates the total volume y_i of a flow f_i prior to its transmission. The model incorporates a temporal context window that captures the current network state and the feature vectors of the K preceding flows.

Flow Characterization. A flow $f_i \in \mathcal{F}$ is defined as a 6-tuple $\{IP_{src}, IP_{dst}, Port_{src}, Port_{dst}, Proto, Gap_{id}\}$, extending the classical 5-tuple with an additional identifier Gap_{id}. The Gap_{id} distinguishes flows that share the same 5-tuple but are separated in time, i.e., belong to different bursts according to an inter-arrival time criterion. For the purpose of prediction, we define f_i as a sequence of T packets $P_i = \{p_{i,1}, p_{i,2}, \ldots, p_{i,T}\}$, where each packet $p_{i,j}$ carries a payload of size $s_{i,j}$.

The total flow size y_i is defined as the sum of the sizes of all packets belonging to the flow until its finalization, where a flow is considered to end when the inter-arrival time between consecutive packets with the same 5-tuple exceeds the time gap Gap_{id}: $y_i = \sum_{j=1}^{T} s_{i,j}$.

Feature Space and Temporal Windowing. Let each flow f_j be represented by a primary feature vector $\mathbf{z}_j \in \mathbb{R}^d$. This vector includes host-level metrics, network telemetry, and, for completed flows, the realized flow size y_j. We define the components as follows:

∘ **Target Flow:** f_i is the flow about to start, represented by its initial feature vector $\mathbf{x}_i$, obtained from $\mathbf{z}_i$ by removing the target variable y_i, i.e., $\mathbf{x}_i \in \mathbb{R}^{d-1} = \mathbf{z}_i \setminus \{y_i\}$. Note that y_i is unknown at prediction time.

∘ **Historical Context:** $\mathcal{H}_i = \{\mathbf{z}_{i-1}, \mathbf{z}_{i-2}, \ldots, \mathbf{z}_{i-K}\}$ represents the sequence of the K most recent flows. Each $\mathbf{z}_{i-k} \in \mathbb{R}^d$ includes its respective ground-truth size y_{i-k}.

The total input representation for the model, referred to as $\mathbf{X}_i$, is obtained by concatenating the current flow features with the flattened features of the K previous flows:

$$\mathbf{X}_i = \mathbf{x}_i \parallel \text{vec}(\mathcal{H}_i) \tag{1}$$

where $\mathbf{X}_i \in \mathbb{R}^{(d-1)+(K\times d)}$. This high-dimensional vector captures the short-term temporal dependencies and workload patterns typical of data center traffic.

In Practice, the feature space can combine application-, host-, and network-level signals, as presented in Section III.

Gradient Boosting Regression. We employ eXtreme Gradient Boosting (XGBoost) [17] to approximate the mapping $h : \mathbf{X}_i \rightarrow \hat{y}_i$. The model is an ensemble of N additive functions:

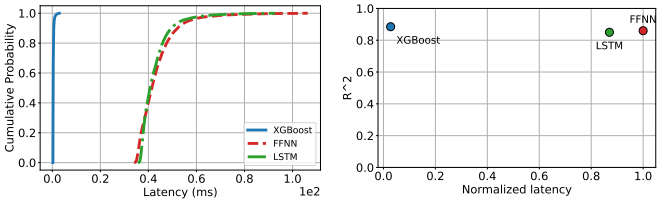
$$\hat{y}_i = \sum_{t=1}^{N} f_t(\mathbf{X}_i), \quad f_t \in \mathcal{N} \tag{2}$$

where $\mathcal{N}$ is the space of regression trees. The learning process minimizes a regularized objective function that combines a loss metric L and a penalty for model complexity Ω:

$$\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_t \Omega(f_t) \tag{3}$$

Given the heavy-tailed nature of flow size distributions, we adopt the Mean Squared Logarithmic Error (MSLE) as the training objective to ensure robustness across multiple orders of magnitude.

$$L(y, \hat{y}) = \frac{1}{2}(\log(y+1) - \log(\hat{y}+1))^2$$



(a) Latency CDF. (b) R^2 and Latency.

Fig. 2: Inference latency at the 99th percentile.

Information Flow and Prediction. By incorporating $\mathcal{H}_i$, the XGBoost model can perform conditional branching based on the performance of past flows. For instance, if the previous K flows from a specific host exhibited high volume and high inter-arrival times, the ensemble of trees can identify this pattern as a specific workload signature and adjust the prediction $\hat{y}_i$ accordingly before the first packet of f_i is even processed.

Why Gradient-Boosted Decision Trees? Following previous work [9], we evaluate three model families for flow-size prediction: feed-forward neural networks (FFNNs), Recurrent Neural Networks with LSTM layers, and GBDTs.

Because our target use case is integration with the transport layer, the predictor must operate with minimal latency. To effectively reduce FCT, the cost of generating predictions should not outweigh the performance gains they enable [18]. Figure 2 compares the inference latency of the evaluated models on the public datasets. We emphasize that these measurements are intended for comparative analysis only, as all models are evaluated under the same hardware conditions on a standard server. While the absolute latency values are not representative of optimized deployments, the results clearly show that the XGBoost implementation of GBDTs achieves significantly lower inference latency than FFNN and LSTM models with comparable accuracy. Tree-based models can be implemented in the kernel with even lower latency [19] and offloaded to the data plane [20], [21], thereby enabling their practical deployment with ultra-low prediction latency.

Cross-Workload Model Construction

A cross-workload model provides a single, robust inference engine capable of simultaneously identifying and adapting to diverse traffic signatures. By training on a heterogeneous dataset comprising multiple workload types, the model learns a broader representation of network behavior. This *global* view allows the system to capture shared underlying patterns, such as the relationship between host CPU state and flow burstiness, which are often ignored by models focused on a single application.

A key challenge in combining datasets from different workloads is the heterogeneity and discrepancy of their feature spaces. Differences in monitoring tools and workload-specific instrumentation lead to partially overlapping feature sets, which result in a sparse representation when datasets are merged. To address this issue, we evaluate different strategies, including (i) *Intersection*: restricting the feature space to those features common to all workloads. While simple, this approach discards potentially informative, workload-specific features; (ii) *Stacking and Ensembles*: training separate base models per workload and combining their outputs through a meta-regressor. This preserves specialization but increases system complexity; (iii) *Union Strategy*: we define a global feature space $\mathcal{X}_{union}$ as the union of all features across datasets, thereby retaining all available information.

The intersection approach is the simplest to implement but suffers from significant information loss, as it forces all workloads into a "lowest common denominator" feature set, often stripping away the high-value, workload-specific metrics that drive accuracy. Stacking and Ensembles should achieve acceptable precision by preserving specialized base learners, but they introduce operational complexity; adding a new workload requires retraining multiple models and a new meta-regressor, making the system difficult to scale. The Union Strategy provides a practical balance. While it creates a higher-dimensional and sparse feature space, it preserves all available information across datasets. The primary advantage is its modularity: when a new workload is introduced, the system simply appends new feature columns. This balance, along with the accuracy limitations observed in models built with the former approaches, is our main motivation for focusing on the Union approach.

The Union Strategy. To extend the formalization to a cross-workload setting, we must address the structural divergence between datasets from diverse applications (e.g., KMeans vs. TensorFlow). We define a global feature space $\mathcal{X}_{union} = \bigcup_{w \in \mathcal{W}} \mathcal{X}_w$, where $\mathcal{W}$ is the set of all considered workloads.

For a flow f_i belonging to workload $w \in \mathcal{W}$, the unified feature vector $\mathbf{X}_i^{union}$ is constructed as follows:

$$\mathbf{X}_i^{union} = \begin{cases} x_{i,j} & \text{if feature}_j \in \mathcal{X}_w \\ \text{NaN} & \text{if feature}_j \notin \mathcal{X}_w \end{cases} \tag{4}$$

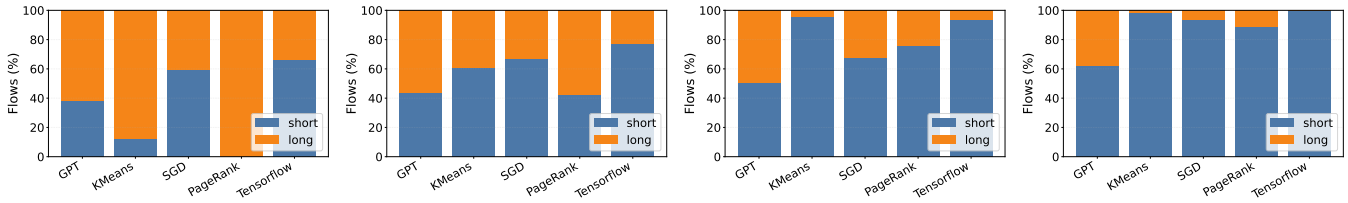
By concatenating this unified vector with the historical window defined before, the input for the model becomes:

$$\mathbf{V}_i = \mathbf{X}_i^{union} \parallel \text{vec}(\mathcal{H}_i^{union}) \tag{5}$$

This formalization relies on XGBoost's sparsity-aware algorithm. Unlike standard regression techniques that require imputation (e.g., mean substitution), the GBDT objective natively handles NaN values by treating them as a separate category. During training, for each split at node v, the model learns a default direction $D_v \in \{(L)\text{eft}, (R)\text{ight}\}$ for missing values that minimizes the training loss:

$$\mathcal{L} = \sum_{i \in I_L \cup \{i \in I_{\text{missing}} : D_v = L\}} L_i + \sum_{i \in I_R \cup \{i \in I_{\text{missing}} : D_v = R\}} L_i \tag{6}$$

where I_{missing} denotes the set of samples with missing feature values. This allows a single global model to act as a specialized predictor for each specific workload w when relevant features are present, while maintaining a baseline prediction when they are absent.



(a) 1KB (b) 10 KB (c) 100 KB (d) 1 MB

Fig. 3: Distribution of short and long flows across AI workloads for four representative short-flow thresholds.

Classification. The same cross-workload strategy can also be applied to flow-size classification. Rather than estimating the continuous size y_i, the model directly predicts whether a flow falls below a given size threshold, which is the decision required by transport mechanisms that differentiate short and long flows. For this, we also formulate flow-size awareness as a binary classification problem. Given a threshold θ that separates short and long flows, each flow f_i is assigned a class label

$$c_i = \begin{cases} 1, & \text{if } y_i > \theta, \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

where $c_i = 1$ denotes a long flow and $c_i = 0$ denotes a short flow. The classifier estimates the probability that a flow belongs to the long-flow class:

$$\hat{p}_i = g(\mathbf{V}_i), \tag{8}$$

where V_i is the union-based representation defined in Equation 5. The predicted class is obtained as

$$\hat{c}_i = \begin{cases} 1, & \text{if } \hat{p}_i \geq \tau, \\ 0, & \text{otherwise,} \end{cases} \tag{9}$$

where τ is the decision threshold.

We train the classifier using XGBoost with a binary logistic objective. The optimization minimizes the regularized binary cross-entropy loss:

$$\mathcal{L}_{\text{cls}} = -\sum_{i=1}^{n} [c_i \log(\hat{p}_i) + (1 - c_i) \log(1 - \hat{p}_i)] + \sum_{t=1}^{N} \Omega(f_t).$$

As in the regression case, the classifier is trained over the global union feature space $\mathcal{X}_{union}$ and relies on XGBoost's sparsity-aware handling of missing values. This allows a single classifier to exploit workload-specific features when they are available, while still producing predictions for workloads where those features are absent.

System Design. The predictor relies on telemetry collected at the end hosts, where application activity, system behavior, and communication events can be observed close to the source of traffic generation. This host-side perspective is important because flow size is not only determined by the network, but also by computation, memory, and storage activity, and previous communication patterns. Accordingly, Table I summarizes the feature groups considered in this work.

TABLE I: Feature groups considered by the models.

Feature Group	Examples
Temporal context	time, previous_flow_time, flow_frequency
Recent flow history	flow_size1..K, previous_flow_time1..K
Host activity	cpu, ram, machine, first_call
Memory and storage	memory_read, memory_write, disk_read, disk_write
Network activity	agg_net_in, agg_net_out, per-machine counters
Transport state	tcp_queue, send/receive-related counters
Feature alignment	workload identifier, missing-feature indicators

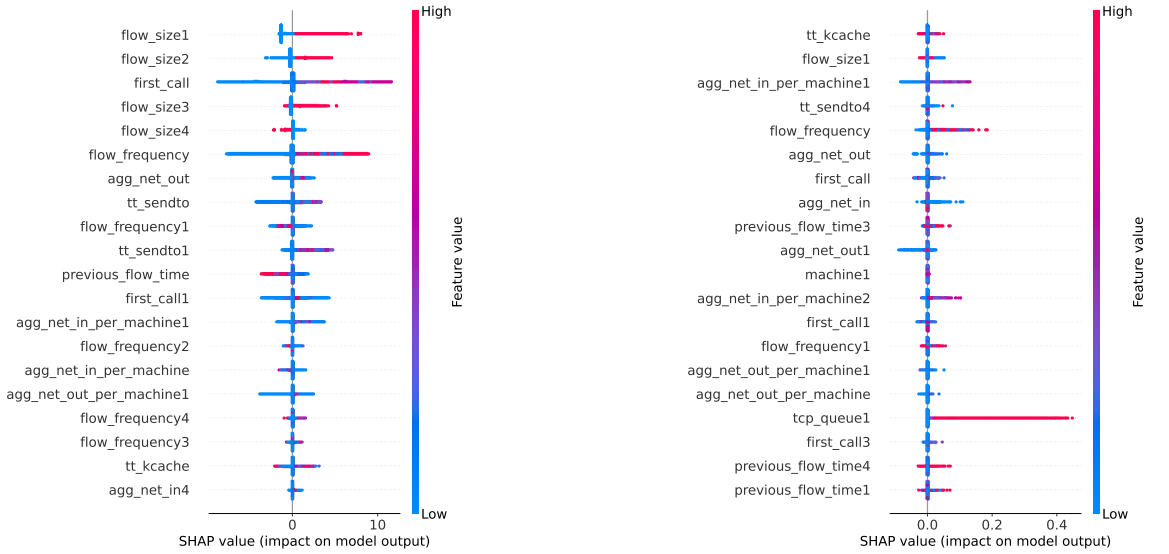
The feature selection aims to preserve predictive information while aligning heterogeneous telemetry spaces. To capture temporal context, each prediction uses the current flow and the previous K flows. The parameter K controls how much recent communication history is exposed to the model. Starting from the Flux [9] recommendation, our validation selects $K = 5$ as the best-performing configuration.

The target variable, `flow_size0`, is excluded from the input. To prevent leakage, all features correspond to observations available before the target flow starts, and historical features use only preceding flows.

Our data combines public and newly collected datasets. We use the Flux dataset [9] as a public baseline because it provides flow-level and system-level instrumentation for distributed AI workloads using CPUs and GPUs at scale. However, Flux reflects an older software and workload environment. Consequently, we extend this dataset with additional traces collected using INTFusion [22]. These traces include an LLM fine-tuning workload based on GPT, as well as updated Spark workloads for KMeans, PageRank, and SGD collected with newer versions of the framework. This combination allows us to compare against a public reference dataset while assessing whether cross-workload prediction remains effective on updated and more representative AI data center workloads.

Model Training. Although the primary prediction task is formulated as regression, to evaluate classification, we used common short flow thresholds of 1 KB, 10 KB, and 100 KB. To further characterize the data, Figure 3 illustrates the class distribution across the evaluated datasets. As observed, the balance is different among the workloads and thresholds.

To open the "black box" of the XGBoost model training, Figure 4 shows the SHAP summary for our models. For the classification task, a positive SHAP value indicates that the feature pushes the model towards predicting Class 1 (long), while a negative value indicates that the feature value pushes the model towards Class 0 (short). In the case of regression,



(a) Classification.

(b) Regression.

Fig. 4: SHAP-based feature attribution for the classification and regression models.

the x-axis directly represents the change in the actual predicted value. As the flow sizes were max-normalized during training, a SHAP value of 0.1, means that a specific feature value pushed the prediction up by 10% of the target range.

The SHAP analysis shows that both models exploit related but not identical signals. For classification, the most influential features are the lagged flow-size variables, indicating that the model primarily relies on temporal locality in recent communication volumes. Features such as *first_call* and *flow_frequency* further suggest that workload phase and communication intensity are important for separating flows across the threshold. In contrast, the regression model relies on host- and network-level signals. This indicates that direct flow-size estimation benefits from micro-level execution signals and aggregate communication context, whereas threshold-based classification is more strongly driven by recent flow-size history.

IV. Evaluation

The evaluation is guided by the following research questions – ***RQ1:*** *To what extent can a single cross-workload regression model replace workload-specific predictors while preserving flow-size prediction quality across heterogeneous AI workloads?* ***RQ2:*** *How effectively can a unified cross-workload classifier distinguish short from long flows for flow-aware transport?* ***RQ3:*** *What is the impact on flow completion time (FCT) of replacing Fork's oracle flow-size knowledge with learned predictions in a realistic transport-layer deployment?*

A. Experimental Methodology & Datasets

We evaluate the proposed predictors from both machine-learning and networking perspectives. At the prediction layer, we assess regression performance using R^2, MAE, and RMSE, and compare the proposed cross-workload predictor against workload-specific models to quantify the cost of replacing per-workload specialization with a unified model. We further evaluate classification performance using accuracy, precision, recall, and F1 score. At the transport layer, we assess the impact of prediction errors on congestion-control decisions by injecting the learned predictions into Fork in ns-3 using the configuration in Table II. Each transport configuration is repeated across independent random seeds, and we report mean results with confidence intervals. Fork provides a strong baseline because it assumes perfect short/long flow identification and has demonstrated improved performance over state-of-the-art DCN transports such as Homa [23].

TABLE II: Simulation setup.

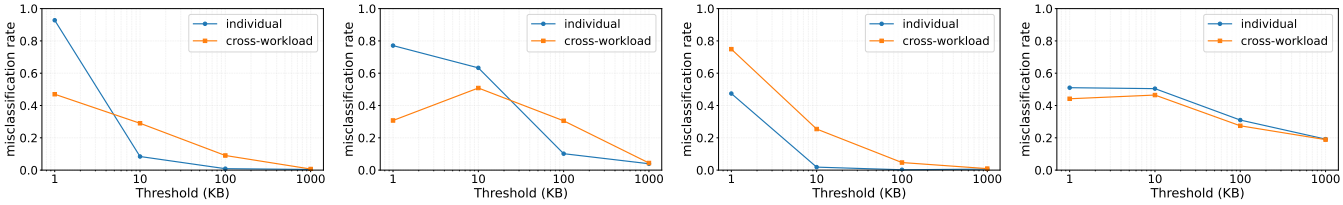
Parameter	Values
Link Speeds	10 Gbps
RTT	60 μs
Buffers	250 KB
Loads	30–90%
Short-flow fraction	80%
Short-flow sizes	1–100 KB
Long-flow sizes	1–10 MB
Flow-size distribution	Empirical DCN distribution
Flow arrival process	Poisson
Short-flow threshold	100 KB

We use the open dataset introduced in [9], comprising four large-scale distributed AI workloads, and complement it with unseen workloads collected on a small university cluster using INTFusion [22], a telemetry architecture for generating new workload traces [11]. We evaluated several XGBoost-based configurations, including feature-intersection and stacking variants; since these consistently yielded lower predictive quality, the evaluation focuses on workload-specific predictors and the proposed cross-workload predictor.

The dataset comprises multiple tabular files per workload, each corresponding to an independent workload run. We split

TABLE III: Regression performance of workload-specific and union models. Positive improvement indicates better performance of the union model (higher R^2 or lower prediction error).

	R^2			MAE (KB)			RMSE (KB)		
Workload	**Specialized**	**Union**	**Imp. (%)**	**Specialized**	**Union**	**Imp. (%)**	**Specialized**	**Union**	**Imp. (%)**
GPT	0.16	**0.18**	**12.5**	1322	**1311**	**0.8**	9428	**9334**	**1.0**
KMeans	**0.88**	0.87	**-1.1**	669	**562**	**16.0**	7739	**7083**	**8.5**
SGD	**0.63**	0.54	**-14.3**	**37**	62	**-67.3**	**316**	350	**-10.8**
PageRank	**0.87**	0.85	**-2.3**	**467**	524	**-12.2**	**4940**	5284	**-7.0**
TensorFlow	**0.97**	0.94	**-3.1**	**1**	3	**-203.5**	**8**	11	**-41.1**



(a) KMeans (b) Page Rank (c) SGD (d) GPT

Fig. 5: Misclassification rate per short-flow threshold.

the data at the file level: every run is assigned entirely to the training, validation, or test set using a 70/20/10 split, preventing flows from the same run from appearing across partitions. Prediction is performed on normalized flow-size values rather than raw byte values: each feature column, including the target, is scaled by the maximum value computed from the training data. Raw flow-size values are reconstructed only after prediction.

B. RQ1: Cross-Workload Prediction Performance

To quantify the predictive cost of replacing workload-specific models with a cross-workload predictor, Table III compares both approaches using R^2, MAE, and RMSE. R^2 captures the fraction of per-workload variance explained by the model, MAE quantifies the typical prediction error, and RMSE places greater weight on large residuals. We additionally report relative differences between the cross-workload and workload-specific models to facilitate comparison across workloads.

The cross-workload regressor remains competitive across heterogeneous workloads, although the effect of unification is workload-dependent. For GPT, the unified model slightly improves all three metrics, and for KMeans it reduces both MAE and RMSE while preserving a comparable R^2. These results indicate that joint training can transfer useful predictive structure across some workloads. In contrast, SGD, PageRank, and TensorFlow exhibit higher absolute errors under the cross-workload model. The largest reduction in R^2 occurs for SGD, suggesting that part of its communication behavior remains workload-specific and is not fully preserved by feature alignment and joint training. TensorFlow remains highly predictable under both configurations, although the workload-specific model achieves lower MAE and RMSE.

The gap between MAE and RMSE indicates that a relatively small number of large residuals contributes disproportionately to squared-error-based metrics. This behavior is consistent with the heavy-tailed nature of flow-size distributions and motivates evaluating prediction quality beyond aggregate regression metrics. Figure 5 therefore reports the threshold-crossing error induced when continuous regression outputs are converted into short/long flow decisions. A threshold-crossing error occurs when the predicted and actual flow sizes lie on opposite sides of the selected boundary. Similar regression performance does not necessarily translate into similar transport behavior, since errors near the short/long decision threshold can change the flow class assigned by the transport protocol, whereas even larger errors away from the threshold may have no transport impact.

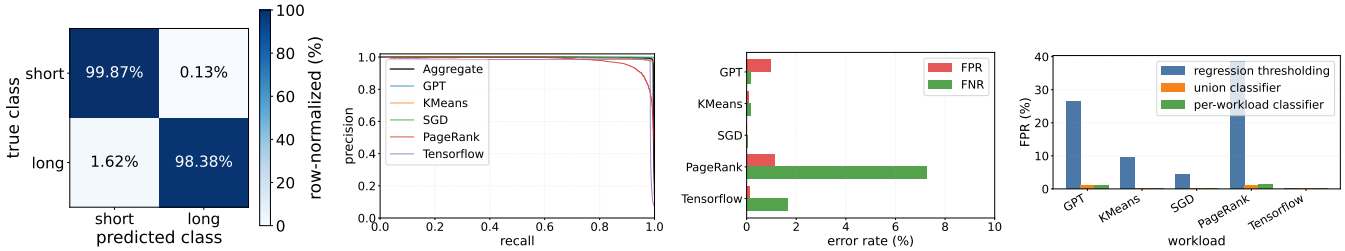
Take-away RQ1

A single cross-workload predictor preserves most of the predictive quality of workload-specific models across heterogeneous AI workloads. Although the cost of unification is workload dependent, the overall degradation remains limited, making a unified predictor a practical alternative to maintaining one specialized model per workload.

C. RQ2: Cross-Workload Classification

Although regression estimates continuous flow sizes, flow-aware transport ultimately requires classifying flows as short or long. Consequently, the most relevant prediction errors are those that cross the decision threshold rather than the largest absolute residuals. We therefore evaluate classification performance using a 100 KB threshold, which provides a representative separation between latency-sensitive short flows and larger transfers while matching the Fork configuration used later.

Figure 6 summarizes the classification performance. The confusion matrix in Figure 6a shows that the cross-workload classifier correctly identifies 99.87% of short flows and 98.38% of long flows. Accordingly, only 0.13% of short



(a) Confusion Matrix (b) Precision-recall (c) FPR and FNR (d) Classification vs. Regression

Fig. 6: Summary of the classification performance metrics.

flows are incorrectly assigned to the long-flow class, while 1.62% of long flows are classified as short. These two error types are particularly relevant for flow-aware transport: false-long decisions may prevent latency-sensitive short flows from receiving preferential treatment, whereas false-short decisions may place large transfers in the short-flow control loop and increase contention for latency-sensitive traffic.

The precision–recall curves in Figure 6b remain close to the upper-right corner for the aggregate test set and most individual workloads, indicating high precision over a broad recall range. PageRank exhibits the largest precision–recall trade-off and the highest false-negative rate, suggesting that its short and long flows are more difficult to separate around the threshold. TensorFlow also shows a non-negligible false-negative rate, whereas KMeans and SGD exhibit near-zero error rates. GPT has the highest false-positive rate, although it remains below 1%.

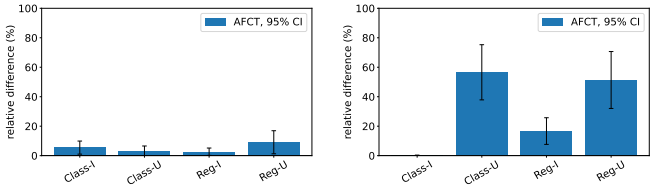
Figure 6d further compares direct classification with regression thresholding. The cross-workload classifier substantially reduces the false-positive rate relative to converting regression outputs into short/long labels, particularly for GPT and PageRank. This result highlights that minimizing continuous flow-size error and minimizing threshold-crossing error are related but distinct objectives. Regression remains useful because it supports arbitrary thresholds and richer transport decisions; however, a classifier trained specifically for the 100 KB boundary can more effectively protect short flows from being incorrectly assigned to the long-flow class.

Take-away RQ2

Despite the limitations of continuous flow-size regression, a unified cross-workload classifier can reliably distinguish short from long flows, achieving over 98% classification accuracy across heterogeneous AI workloads. For flow-aware transport, the most relevant errors are those that cross the short/long decision boundary, particularly underestimating long flows, which has a disproportionate impact on transport performance.

D. RQ3: Transport-Layer Validation

Finally, we evaluate whether learned flow-size prediction can effectively replace Fork's oracle flow-size knowledge in a realistic flow-aware transport protocol. To this end, we integrate the workload-specific and cross-workload models (m) into Fork running in ns-3 under trace-driven traffic and quantify their impact using the relative average flow completion time (AFCT) difference with respect to the original Fork configuration, which assumes perfect short/long flow identification:



(a) GPT (b) KMeans

Fig. 7: AFCT relative difference using real traces in Fork.

$$\Delta_{\mathrm{AFCT}}(m) = \frac{\mathrm{AFCT}m - \mathrm{AFCT}_{\mathrm{Fork}}}{\mathrm{AFCT}_{\mathrm{Fork}}} \times 100.$$

A value of 0% indicates performance identical to the Fork-oracle baseline, whereas positive values represent the AFCT penalty incurred by replacing perfect flow-size knowledge with learned predictions. This normalization enables meaningful comparison across workloads with different traffic characteristics and baseline FCTs.

Figure 7 reports Δ_{AFCT} for four prediction settings: workload-specific and cross-workload classifiers (Class-I and Class-U), and workload-specific and cross-workload regressors (Reg-I and Reg-U). Real traces from the test partition are replayed as foreground traffic, while the statistical workload of Table II provides background traffic.

The transport impact of model generalization is workload dependent. For KMeans, the workload-specific classifier remains closest to the Fork-oracle baseline, whereas the cross-workload classifier and regression-based variants incur larger AFCT penalties. In contrast, GPT exhibits only minor differences across all prediction approaches, with both classifiers remaining close to the oracle configuration and the regression models introducing only limited degradation. These results are consistent with the prediction-layer analysis: cross-workload learning preserves transport performance when workload patterns transfer well, whereas degradation becomes visible when threshold-crossing errors alter Fork's short/long

flow assignment. More generally, transport behavior cannot be inferred from aggregate prediction accuracy alone. Predictors with similar regression or classification metrics may produce different AFCTs if their remaining errors occur near the decision threshold or systematically classify long flows as short. Conversely, modest reductions in predictive accuracy have little transport impact when most prediction errors remain on the correct side of the threshold.

Finally, to better understand the transport results, we independently vary short- and long-flow classification accuracy while measuring the resulting AFCT. Figure 8 reports the relative AFCT penalty with respect to the Fork-oracle baseline, where the horizontal and vertical axes represent long- and short-flow prediction accuracy, respectively.

The results reveal a strong asymmetry between the two error types. Long-flow accuracy below approximately 90–95% frequently more than doubles AFCT, regardless of short-flow accuracy. In contrast, once long-flow accuracy reaches 98–99%, AFCT remains close to the oracle baseline over a broad range of short-flow accuracies. This behavior reflects Fork's higher sensitivity to underestimating large flows, which incorrectly routes them through the short-flow control loop and increases contention for latency-sensitive traffic.

Conversely, overestimating short flows has a smaller and less systematic impact, as it only prevents preferential treatment without introducing large transfers into the short-flow path. These results explain the workload-dependent AFCT observed previously and identify long-to-short misclassification as the dominant source of transport degradation.

Take-away RQ3

Integrating the proposed predictors into Fork shows that learned flow-size prediction can effectively replace oracle flow-size knowledge with only limited transport degradation. The unified cross-workload predictor preserves most of the transport benefits of workload-specific models while eliminating the need for one predictor per workload. Transport performance is primarily determined by long-to-short misclassification, highlighting the importance of minimizing threshold-crossing errors.

V. Conclusion

This paper presents a unified cross-workload flow-size prediction framework for AI data center networks that aligns heterogeneous end-host telemetry and eliminates the need for workload-specific predictors. Our results show that a single model preserves most of the predictive quality and transport benefits of specialized models, establishing cross-workload flow-size prediction as a practical foundation for learning-assisted congestion control without requiring oracle flow-size knowledge or one model per workload. More importantly, we show that transport-aware evaluation is essential: similar regression performance can lead to markedly different transport behavior, whereas minimizing threshold-crossing errors and preserving long-flow identification translate more directly into improved flow completion times. We hope these findings

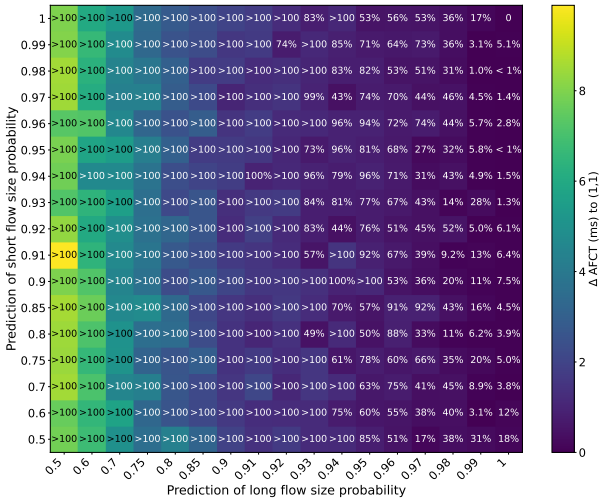


Fig. 8: Short and long flow sensitivity analysis over an oracle.

encourage closer co-design of machine learning models and transport protocols for future AI data center networks.

References

[1] M. Noormohammadpour et al., "Datacenter Traffic Control: Understanding Techniques and Tradeoffs," *IEEE Comm. S. & T.*, vol. 20, pp. 1492–1525, 2018.

[2] S. Joy and A. Nayak, "Improving flow completion time for short flows in datacenter networks," in *IFIP/IEEE IM*.

[3] Y. Liu *et al.*, "Fork: A Dual Congestion Control Loop for Small and Large Flows in Datacenters," in *EuroSys*, 2025.

[4] L. Alberro and E. Grampin, "Transport protocols in the data center: Understanding proposals and research challenges," *Computer Networks*, vol. 274, p. 111793, 2026.

[5] M. Li *et al.*, "ScaleAcross Explorer: Exploring Communication Optimization for Scale-Across AI Model Training," 2026. [Online]. Available: https://arxiv.org/abs/2605.24326

[6] A. Roy *et al.*, "Inside the Social Network's (Datacenter) Network," in *SIGCOMM'15*, 2015.

[7] T. Benson *et al.*, "Understanding data center traffic characteristics," *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 1, p. 92–99, Jan. 2010.

[8] A. R. Curtis et al., "Mahout: Low-overhead datacenter traffic management using end-host-based elephant detection," in *INFOCOM*, 2011.

[9] V. Dukić *et al.*, "Is advance knowledge of flow sizes a plausible assumption?" in *NSDI*, 2019.

[10] S. M. Hosseini *et al.*, "Flow Size Prediction with Short Time Gaps," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2024, pp. 01–07.

[11] L. Alberro *et al.*, "Unified Learning Repository," https://github.com/leoalb/unified-learning, 2026, accessed: 2026-06-29.

[12] G. Wang et al., "c-Through: part-time optics in data centers," in *SIGCOMM '10*, 2010.

[13] A. Bayati *et al.*, "Gaussian process regression ensemble model for network traffic prediction," *IEEE Access*, vol. 8, pp. 176 540–176 554, 2020.

[14] X. Ma et al., "Cellular network traffic prediction based on correlation convlstm and self-attention network," *IEEE Comm. Lett.*, vol. 27, pp. 1909–1912, 2023.

[15] M. Li *et al.*, "A deep learning method based on an attention mechanism for wireless network traffic prediction," *Ad Hoc Networks*, vol. 107, 2020.

[16] P. Poupart *et al.*, "Online flow size prediction for improved network routing," in *IEEE ICNP*, 2016.

[17] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *ACM KDD*, 2016.

[18] G. Wan *et al.*, "CATO: End-to-End optimization of ML-Based traffic analysis pipelines," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. Philadelphia, PA: USENIX Association, Apr. 2025, pp. 1523–1540.

[19] DMLC Contributors, "Treelite: Universal Model Exchange and Serialization Format for Decision Tree Forests," https://github.com/dmlc/treelite, 2026.

[20] C. Zheng *et al.*, "Planter: Rapid Prototyping of In-Network Machine Learning Inference," *SIGCOMM Comput. Commun. Rev.*, vol. 54, p. 2–21, Aug. 2024.

[21] B. Brandino *et al.*, "Detecting Attacks at Switching Speed: Ai/Ml and Active Learning for in-Network Monitoring in Data Planes," in *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*, 2024, pp. 1–6.

[22] L. Alberro *et al.*, "INTFusion: Unifying Network and Host Telemetry in Data Center Networks," in *IFIP Networking*, 2026.

[23] B. Montazeri *et al.*, "Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities," in *SIGCOMM'18*, 2018.